

OG InsaneSDD-2.0 Demo

Video length: Approximately 2 minutes 37 seconds **Audio:** No audio track was present, so this report is based on the visible interface and actions.

Software overview

The demo shows **InsaneSDD by InfoBeans**, an AI-assisted software development orchestration platform.

It converts a plain-text product idea into:

Requirements → approved baseline → UX specification → epics/stories/tasks → repository architecture → automated development → Git branches, commits and pull requests → progress monitoring

The example project created in the demo is **AgentPilot**, an autonomous AI task-management platform.

Screen-by-screen information

1. Projects and Create Project screen

Purpose: Create and manage software projects.

Information entered:

- **Project name:** AgentPilot
- **Description:** An AI-native operations platform that accepts goals in plain text, creates AI agents, assigns roles, generates task breakdowns, estimates timelines and tracks progress.

Available actions:

- Create project
- Cancel
- Open existing projects
- Delete projects
- Create another new project

The project initially receives the status **Spec Created**.

2. Project Overview screen

Purpose: Provide the project status, workflow modules and integrations.

Main project modules shown:

- **Spec Panel:** Requirements input, AI analysis and approved baseline
- **UX Design:** AI-generated personas, journeys, screens and design system
- **Agile Editor:** Epics, stories and task hierarchy
- **Kanban Board:** Task workflow tracking
- **Live Terminal:** Real-time execution activity
- **Branches:** Branch explorer and pull-request status
- **Change Requests:** Change impact analysis and versioning
- **Deploy:** Deployment workflow

GitHub integration:

- Connected to a GitHub App installation
 - Connected account: jamesolinone
 - Three repositories available
 - Supports branch, pull-request and webhook synchronization
-

3. Requirement Specification screen

Purpose: Convert an unstructured product description into an approved requirement specification.

The screen has three panels:

Raw Input

Accepts a project description in plain text.

The AgentPilot input describes:

- AI agents with planner, executor and reviewer roles
- Automatic task decomposition
- Dependency mapping and priority scoring
- Gantt-style timeline generation
- Deadline-risk prediction
- Automatic schedule adjustment
- Real-time progress and log tracking
- Agent handoff, retry and fallback
- Human override controls
- Audit trails and agent cost tracking
- Role-based access control
- APIs, webhooks and MCP integrations
- JSON, Markdown and PDF report export

AI Understanding

Displays the structured interpretation produced by AI.

Frozen Baseline

Stores the approved and versioned specification.

Each panel can be maximized for detailed review.

4. AI Understanding review screen

Purpose: Review and edit the AI-generated interpretation before approving it.

Generated title:

AgentPilot Autonomous Task Management Platform

Generated requirement sections include:

- Summary
- Objectives
- Assumptions
- Out-of-scope items
- Proposed technology stack
- Reasoning behind the selected stack

Example objectives:

- Convert plain-text goals into discrete tasks
- Assign AI-agent roles
- Build a dependency-aware Gantt timeline
- Score priorities and predict deadline risks
- Track agent progress and logs in real time
- Support agent handoff, retry and fallback
- Allow human override
- Maintain audit trails, cost tracking and access control

Generated technology stack:

Layer	Technologies
Frontend	React, Redux, D3.js
Backend	Node.js, Express, Python
Database	PostgreSQL, Redis
Infrastructure	Kubernetes, Docker, AWS EC2, S3, Lambda, API Gateway

Available actions:

- Edit generated information
- Add or delete objectives
- Save changes

- Approve and freeze baseline
 - Reject the interpretation
-

5. Frozen Baseline screen

Purpose: Preserve the approved requirements as a controlled project baseline.

The approved specification is frozen as **version 1** and contains:

- Project title and summary
- Objectives
- Assumptions
- Out-of-scope items
- Technology stack
- Architecture reasoning

This baseline becomes the source for UX design and agile-plan generation.

6. UX Design initialization screen

Purpose: Generate the product's UX specification from the approved baseline.

Before generation, the screen displays:

No UX Design yet

The **Run UX Design Agent** action starts AI generation. The interface states that generation may take approximately 30 to 60 seconds.

7. Generated UX Design screen

Purpose: Review a complete AI-generated UX plan before development planning.

The generated design contains approximately:

- **3 personas**
- **3 user journeys**
- **5 screens**
- **30 components**
- **5 accessibility requirements**
- **4 performance considerations**

Personas shown

- **Alex, Project Manager**
- **Jordan, AI Operations Specialist**
- **Taylor, Compliance Officer**

Each persona includes:

- Role and technical proficiency
- Responsibilities
- Goals
- Frustrations
- Device preferences

Proposed application screens

- Dashboard
- Task Detail
- Workflow Templates
- Reports Export
- Login

Navigation design

Desktop: Sidebar navigation **Mobile:** Hamburger-menu navigation

Primary navigation:

- Dashboard
- Tasks
- Agents
- Workflows
- Reports

Secondary navigation:

- Settings
- Integrations
- Audit Logs
- Help

Authentication flow:

- Login
- Forgot Password

8. UX Design System and standards

The generated UX specification also contains:

Design system

- Colour palette
- Typography hierarchy
- Spacing scale
- Layout rules

Component inventory

Components for:

- Layout
- Forms
- Navigation
- Data display
- Feedback
- Overlays
- Charts

Accessibility

Requirements shown include:

- WCAG 2.1 AA colour contrast
- ARIA roles and labels
- Text alternatives for non-text content
- Screen-reader compatibility for live updates
- Accessible dynamic controls

Responsive strategy

A mobile-first design that changes from stacked layouts on smaller screens to multi-column or grid layouts on larger screens.

Performance considerations

- Lazy-load logs and timeline information
- Debounce validation and API requests
- Cache frequently used templates and reports
- Optimize SVG and icon assets

Internationalisation

- Multiple-language support
- Locale-based date, time and number formatting
- Right-to-left layout support
- Localized model configuration by agent role

Approval controls:

- Approve UX Design
- Re-run UX Agent
- Request Revision

9. Agile Editor and spec-driven planning screen

Purpose: Convert the approved requirements and UX specification into an executable development plan.

The platform prepares a **monorepo-oriented spec-driven plan** before execution.

Generated estimates:

Metric	Value
Epics	10
Stories	16
Tasks	41
Story points	118
Estimated hours	275

Available actions:

- Prepare spec-driven plan
- Force re-prepare
- Refresh status
- Approve all and execute

This is the final human approval gate before automated development begins.

10. Initial GitHub Repository screen

Purpose: Initialize the project repository.

A private GitHub repository is created:

```
insane-sdd-agentpilot-200654
```

Initial contents include:

- `.gitignore`
- `README.md`

The README states that the repository was initialized by InsaneSDD and that the spec-driven preparation process will generate architecture documentation and the folder structure.

11. Agile execution progress screen

Purpose: Monitor automated task execution.

The screen tracks:

- Total tasks
- Branches created
- Tasks committed

- Pull requests opened
- Failed tasks

Each task row shows:

- Task title
- Related epic and story
- Branch availability
- Commit status
- Pull-request status
- Execution status

Example generated tasks include:

- Implement goal-parsing service
- Assign AI-agent roles
- Create a task-generation API
- Develop the goal-input and task-review interface
- Implement webhook listeners
- Develop report-generation services

During the demo, the counters progress from one active branch to:

- Six branches
- Five committed tasks
- Five opened pull requests
- Zero failures

12. Kanban Board screen

Purpose: Visualize the generated development workflow.

Workflow columns shown include:

- Backlog
- AI Ready
- Analysis
- Approval
- Development
- Review
- QA
- Validation
- UAT
- Done

Each task card contains:

- Task title
- Epic or story
- Current status
- Estimated hours

- Repository and branch
- Pull-request number
- Merge status

The board can be refreshed, horizontally navigated and opened in fullscreen mode.

The demo shows tasks moving from the backlog into development and then into **Done**, with merged pull requests.

13. Live Action Terminal screen

Purpose: Display the real-time autonomous development pipeline.

The terminal records events such as:

- AI-generated epics, stories and tasks
- Workflow state transitions
- Branch creation
- Development execution
- QA status
- Model used for execution
- Commit creation
- Pull-request opening
- Successful execution
- Pull-request merging

The demo shows executions using `gpt-4.1-mini`.

Controls:

- Pause live output
- Fullscreen mode

This screen provides an operational audit trail of automated development activities.

14. Branch Explorer screen

Purpose: Review generated branches and pull-request activity.

The demo briefly opens the Branch Explorer before navigating to GitHub.

The module is intended to connect project tasks with:

- Feature branches
- Commits
- Pull requests
- Merge status

The internal branch list did not remain visible long enough in the recording for deeper review.

15. GitHub Pull Request screen

Purpose: Validate task-level traceability between InsaneSDD and GitHub.

The demonstrated pull request is:

[Dynamic Gantt Timeline and Scheduling] Design data model for tasks with dependencies and scheduling metadata #5

The pull request includes:

- Task description
- Associated story
- Associated epic
- Feature branch
- Files changed
- Commit information
- Labels
- Merge status

It was automatically merged into the main branch and marked as generated by InsaneSDD.

16. Generated Repository and README screen

Purpose: Show the resulting project structure and implementation mapping.

The final repository includes folders such as:

- agentpilot
- backend
- docs
- frontend
- src
- tests

By the final screen, the repository shows:

- Seven branches
- Nine commits
- Generated source code and documentation

Environment variables

- DATABASE_URL
- NEXT_PUBLIC_API_URL
- PORT

Documentation references

- `docs/specs/v1/architecture.md`
- `execution.md`

Task mapping

The README maps individual development tasks to:

- Application modules
- Feature branches
- Generated implementations

Examples include:

- Goal parsing
- AI-agent role assignment
- RBAC schema and middleware
- Agent-handoff and failure recovery
- Audit logging and cost tracking
- External integration management

The repository language breakdown shown is approximately:

- TypeScript: 66.6%
- Python: 20.5%
- CSS: 12.9%

End-to-end workflow demonstrated

```
Create project
  ↓
Submit plain-text requirements
  ↓
Generate and review AI understanding
  ↓
Approve and freeze specification
  ↓
Generate and approve UX design
  ↓
Generate epics, stories and tasks
  ↓
Prepare repository architecture
  ↓
Approve automated execution
  ↓
Create branches, code, commits and pull requests
  ↓
Merge completed work
```

↓
Track everything through Agile Editor, Kanban and Live Terminal

Core value shown in the demo

InsaneSDD is presented as a **human-supervised, AI-driven software delivery system**. It does not immediately execute from an unreviewed prompt. It introduces approval gates for:

- Requirement interpretation
- Requirement baseline freezing
- UX design approval
- Task and execution approval

After approval, it automates the technical delivery workflow while maintaining traceability through tasks, branches, commits, pull requests and live logs.